

EXCEPTION HANDLING IN PYTHON

1. Try ..except block

```
1 try:
2     print(x)
3 except:
4     print("An exception occurred")
```

Output:

```
An exception occurred
```

2. ZeroDivisionError Exception

```
1 def divide(x, y):
2     try:
3         print(f'{x}/{y} is {x / y}') # f is for f(x)=>function in math
4     except ZeroDivisionError as e: # e = caught exception in python
5         print(e)
6
7 divide(10,10)
8
9 divide(10, 4)
```

Output:

```
10/10 is 1.0
10/4 is 2.5
```

Now, add code as below:

```
10
11 divide(10, 0)
```

Output:

```
division by zero
```

3.

```
1 def divide(x, y):
2     try:
3         print(f'{x}/{y} is {x / y}')
4     except (ZeroDivisionError, TypeError, ValueError, NameError) as a: # e = caught exception in python
5         print(a)
6 divide (5,0)
7 divide (2,5)
```

Output:

```
division by zero
2/5 is 0.4
```

4. NameError Exception

```
1 try:
2     print(x)
3 except NameError:
4     print("Variable x is not defined")
5 except:
6     print("Something else went wrong")
```

Output:

5. try..except..else

```
1 try:
2     x=float(input("Enter your number:"))
3     result=100/x # 100/50=2
4     print(result)
5 except Exception as e:
6     print("exception occurred:",e) # e = caught exception in python
7 else:
8     print("else block is executing")
```

Run the code and insert number **10**, observe the output. Then run again and insert number **0**.

Enter your number:

Output 1:

```
Enter The Number:50  
2.0  
else block is executing
```

Output 2:

```
Enter The Number:0  
exception occured: float division by zero
```

6. Nested Exception

```
1  x = 10  
2  y = 0  
3  
4  try:  
5      print("outer try block")  
6      try:  
7          print("nested try block")  
8          print(x / y)  
9      except TypeError as te: #te=caught exception in python  
10         print("nested except block")  
11         print(te)  
12 except ZeroDivisionError as ze: #ze=caught exception in python  
13     print("outer except block")  
14     print(ze)
```

Output:

```
outer try block  
nested try block  
outer execept block  
division by zero
```

7.

```
1 ▾ def divide(x, y):  
2     try:  
3         print(f'{x}/{y} is {x / y}')# {} = barren dictionary in python  
4     except ZeroDivisionError as e: # e = caught exception in python  
5         print(e)  
6     else:  
7         print("Divide() function worked fine.")  
8 ▾ finally:  
9     print("Close all the resources here")  
10    print()
```

Then continue type the segment code as below:

```
12 divide(10, 2)
```

Output 1:

```
10/2 is 5.0  
Divide() function worked fine.  
Close all the resources here
```

```
13 divide(10, 0)
```

Output 2:

```
division by zero  
Close all the resources here
```

```
14 divide(10, "c")
```

Output 3:

```
TypeError: unsupported operand type(s) for /:  
'int' and 'str'
```

8.

```
1  try:
2      num = int(input("Enter the number: "))
3      num1 = int(input("Enter divisible number:"))
4      re = num/num1#15/3=5
5  except ValueError:
6      print("Value is not int type")
7  except ZeroDivisionError as e:
8      print("Don't use zero", e) # e = caught exception in python
9  else:
10     print("result is ",round(re,3)) #round use to round up number
```

Run the code and insert input as below:

```
Enter the number: 15
Enter divisible number:g
```

Output 1:

```
Value is not type
```

```
Enter the number: 15
Enter divisible number:4
```

Output 2:

```
Enter The Number : 15
Enter divisible number : 4
result is  3.75
```

9. Raising Exception

```

1  class ValidationError(Exception):
2      pass
3
4
5  def divide(x, y):
6      try:
7          if type(x) is not int:
8              raise TypeError("Unsupported type")
9          if type(y) is not int:
10             raise TypeError("Unsupported type")
11     except TypeError as e:
12         print(e)
13         raise ValidationError("Invalid type of arguments")
14
15     if y is 0:
16         raise ValidationError("We can't divide by 0.")
17
18
19  try:
20     divide(10, 0)
21  except ValidationError as ve:
22     print(ve)

```

Run and observe the output

Output:

```
We can't didivde by 0
```

Now change line no 20 as below and run.

```

19  try:
20     divide(10, "5")
21  except ValidationError as ve:
22     print(ve)

```

Output:

```
Unsupported type
Invalid type of arguments
```

10. Raising Exception

```
1 class InvalidRange(Exception):
2     pass
3 try:
4     marks=input("Enter the marks:")
5     marks=int(marks)
6     if(marks<0 or marks>100):
7         raise InvalidRange
8     print("Marks =", marks)
9
10 except ValueError:
11     print("Invalid Input")
12 except InvalidRange:
13     print("Input value out of range")
```

Enter the marks:k

Output:

Invalid Input

Enter the marks:200

Output:

Input value out of range